

Data Science Capstone Project Final Report: Model Comparison and Fairness Audit

Course: MSDS640 Data Science Capstone · Dataset: UCI Adult Census Income (1994 U.S. Census) · Erik Herb

Abstract

This project predicts whether a person earns more than 50K a year from census demographic, education, and job information, framed as a lender or screening tool that flags likely high earners. I build three models on the UCI Adult Census Income data, a logistic regression baseline, an XGBoost gradient boosted tree model, and a multilayer perceptron, all trained inside one pipeline that scales the numeric columns and applies SMOTE to training rows only. I compare them on a held-out test set and on five stratified cross-validation folds, and I back every comparison with a statistical test and not just an eyeballed difference, using paired t-tests and Wilcoxon signed-rank tests on the same folds, McNemar's exact test on the same test rows, and paired bootstrap confidence intervals. The project tests three hypotheses. Hypothesis 1 is that education level drives the over 50K prediction more than hours worked, and permutation importance on all three trained models supports it with confidence intervals that exclude zero. Hypothesis 3 is that XGBoost beats both the logistic baseline and the MLP on PR-AUC and F1, and it is supported, with every paired test significant. Hypothesis 2 is that XGBoost is the fairest of the three across nativity, with the smallest US-born versus foreign-born recall gap, and it is refuted on this dataset: XGBoost has the largest gap of the three, and no difference between models is statistically distinguishable at this sample size. The accuracy-first model is not the fairness-first model here, which is the most useful thing the project found.

1. Introduction and problem statement

I predict whether a person earns more than 50K a year from their demographic, education, and job information. The use case I frame the whole project around is a lender or screening tool that flags likely high earners, because that framing decides which errors matter. Under it a missed high earner is a lost customer for the lender and a denied opportunity for the applicant, while a false alarm is a wasted review, so I care about recall on the positive class and about which features drive the decision, not about raw accuracy. Accuracy is actively misleading here, because a model that predicts nobody earns over 50K is already right about 76 percent of the time while catching zero real high earners.

This matters because income screening models are used in lending, credit limits, insurance pricing, and marketing, so the same structure shows up in decisions that affect people's access to money. The lender benefits from catching more real high earners. The applicant benefits from a model that does not quietly work worse for the group they belong to. That second point is why a fairness audit sits next to the accuracy comparison in this project instead of being tacked on at the end, and on this data that decision turned out to matter, because the most accurate model was not the fairest.

For background, the Adult Census Income data is one of the standard benchmark datasets for tabular binary classification, and it is also one of the standard datasets in the algorithmic fairness literature, because it carries sex, race, and native country alongside an income label. Gradient boosted trees are the usual strong performer on tabular data of this size, which is where my expectation for Hypothesis 3 comes from. The fairness side is where I did not have a settled expectation, and that is what Hypothesis 2 tests.

2. Research questions and hypotheses

RQ1 asks which feature drives the over 50K prediction more, education level or hours worked. Hypothesis 1 is that education level is the stronger driver. This is testable because I can measure how much each feature contributes to a trained model with permutation importance, put a confidence interval on the difference between the two, and see whether that interval excludes zero.

RQ2 asks which of the three models is fairest across nativity toward true high earners. Hypothesis 2 is that XGBoost is the fairest, with the smallest US-born versus foreign-born recall gap on real high earners, smaller than logistic regression and the MLP. This is testable because recall on real high earners can be computed separately for each nativity group, the gap between the two is a single number per model, and a paired bootstrap gives a confidence interval on the difference between any two models' gaps. The foreign-born group is about 9 percent of the data, so this is a question about how each model behaves on a group it sees far less of.

RQ3 asks which of the three models classifies best on the held-out data. Hypothesis 3 is that XGBoost beats both the logistic regression baseline and the MLP on PR-AUC and F1. The reasoning behind this is specific, not a guess that the fancier model wins. My EDA found clear non-linear structure in this data: the over 50K rate rises through the working years and falls at the oldest ages, having any capital gain flips the rate sharply instead of gradually, and education and marital status matter together instead of separately. Boosted trees capture curves, thresholds, and interactions by splitting, which is what that structure calls for, while logistic regression only gets them if I add them by hand. The size of the dataset points the same way. About 32,500 rows is small for a neural network, which usually needs far more data before it can learn those interactions from scratch, and it is comfortably enough for a boosted tree ensemble. So the dataset is not large enough to favor the MLP and the structure is not linear enough to favor logistic regression, which is why I expect XGBoost to land in the middle of those two constraints and win. This is testable because PR-AUC and F1 can be computed on the same held-out rows and the same cross-validation folds for all three models, and paired tests say whether the differences are larger than fold-to-fold noise.

3. The data

The data is the Adult Census Income dataset, extracted by Barry Becker from the 1994 U.S. Census Bureau database and hosted by the UCI Machine Learning Repository. It was collected by the Census Bureau as a population survey, and the extract keeps records that pass a set of filters, mainly age over 16 and a positive reported income. It has 32,561 rows and 15 raw columns, one row per person, and after cleaning it is 32,537 rows and 13 columns plus the engineered ones I add. The time range is a single snapshot, the 1994 census year, so there is no time dimension, nothing here is a time series, and no temporal split applies. The dataset is public and released for research use under the UCI repository terms with no access restrictions, and it is anonymized at source with no names or identifiers. The target is income, either over 50K or 50K and under, and it is imbalanced at about 24 percent high earners.

As in every earlier milestone, the raw CSV is loaded into a SQLite database once and pulled back out with a SQL query before any analysis, so the database is the single source of truth for the whole project.

4. Exploratory findings that carry into the models

I am not repeating the whole EDA milestone here. Four findings are what the rest of this report depends on, so I carry those forward and leave the rest behind.

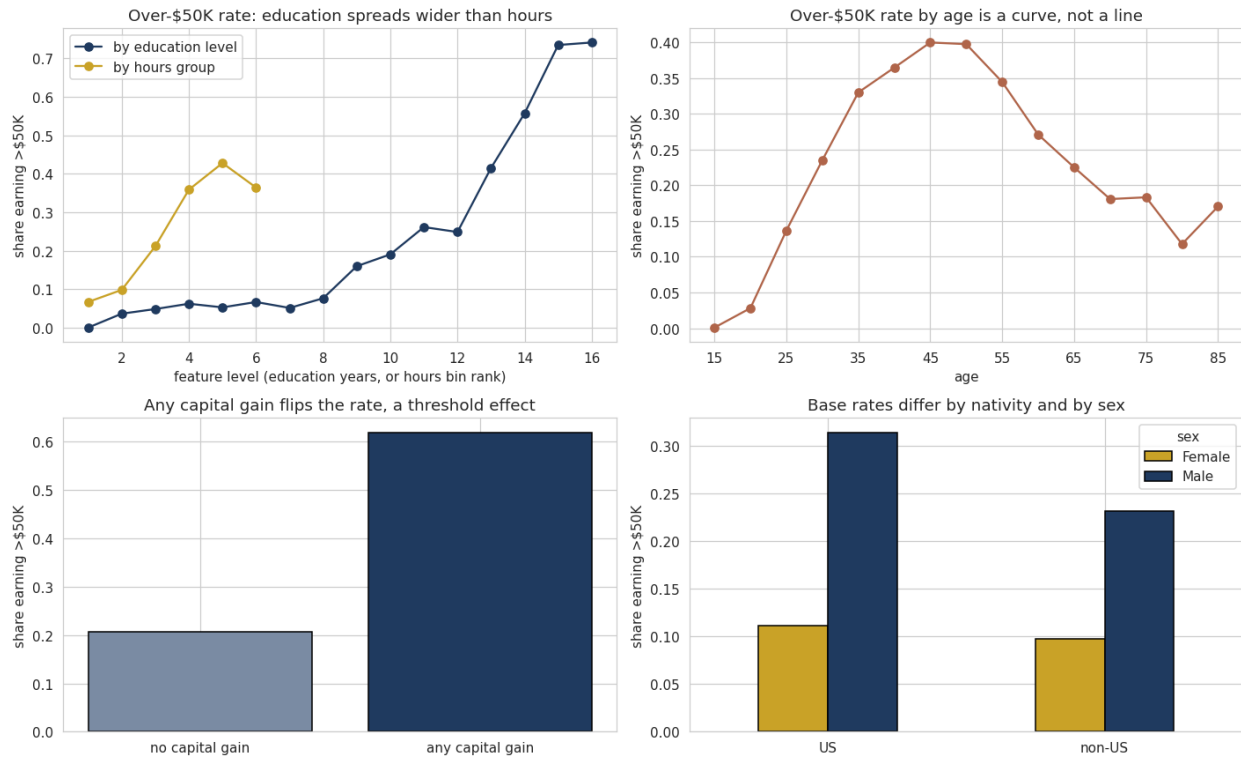


Figure 1. The four EDA findings the hypotheses rest on. Top left is the Hypothesis 1 hint, top right and bottom left are the non-linear structure behind Hypothesis 3, and bottom right is the group base-rate gap Hypothesis 2 is measured against.

The first is the Hypothesis 1 hint. Education produces a spread of about 0.74 in the over 50K rate from its lowest to its highest level while hours produces about 0.36, and education climbs steadily while hours flattens above full time. This is one feature at a time, so it is a hint, not a test, and the real test comes from permutation importance on the trained models in section 21.

The second and third are the Hypothesis 3 reasoning. The over 50K rate by age rises through the working years and falls at the oldest ages, which a linear model cannot express without me adding an age-squared term by hand, and having any capital gain at all flips the rate from low to very high, which is a threshold, not a gradual slope. Threshold effects and curves are exactly what tree splits capture.

The fourth sets up Hypothesis 2. US-born people earn over 50K at about 24.6 percent against 18.7 percent for foreign-born, and men are higher than women inside both nativity groups. I confirmed in the EDA milestone that the nativity base-rate gap is real, with a 95 percent interval of 4.4 to 7.4 percentage points. Those base-rate gaps are what the models learn from, so a recall gap later cannot be blamed entirely on the model.

5. Data preprocessing

Only three columns are missing anything: workclass at 5.64 percent, occupation at 5.66 percent, and native country at 1.79 percent, all categorical. I fill each with an explicit Unknown category instead of dropping the rows. Dropping every row with a missing value would throw away roughly 2,400 people, and the missingness is not random, so dropping could bias the income analysis. An Unknown bucket keeps the rows and lets the model treat missingness as its own signal.

I drop the 24 exact duplicate rows while the sampling weight is still in the table, on purpose, because that weight is close to unique per row and dropping it first would make genuinely different people look identical and get removed by mistake. I then drop the sampling weight, which is a census weight, not a fact about the person, and the text education column, which repeats the numeric one.

I keep every flagged outlier. I measured what dropping them would cost in the EDA milestone instead of arguing it from principle: the IQR rule flags 39.7 percent of the data and 47.2 percent of all high earners, and the flagged rows are signal, not noise, since rows flagged on capital gain are 61.8 percent high earners against 20.7 percent for everyone else. Dropping them would also skew the fairness audit, because foreign-born people are flagged at 45.3 percent against 39.2 percent for US-born, so I would be measuring a nativity gap on a sample I had already skewed by nativity. The hours flag rate of 27.7 percent is an artifact of the rule, not evidence of bad data: 46.7 percent of people work exactly 40 hours, which squeezes the interquartile range to 5 and puts the fence at 32 to 52 hours, so somebody working 55 hours gets called an outlier. Working 55 hours a week is not an anomaly, it is a job.

I one-hot encode the eight categorical columns and drop the first level of each to avoid a redundant column, which gives a 52 column model matrix. I standardize the five continuous numeric columns and pass the binary dummies and flags through untouched, because scaling a 0/1 column does nothing useful. Scaling matters for logistic regression and the MLP, which are sensitive to feature units, and is harmless for XGBoost, which splits on order. The scaler is fit inside the pipeline so it only ever sees training rows.

The target is about 75.9 percent under 50K and 24.1 percent over, roughly 3.2 to 1. I oversample the minority class with SMOTE on training rows instead of undersampling the majority, because undersampling would throw away most of the real under 50K rows while oversampling keeps every real row.

6. Feature engineering and dimensionality reduction

I add four features, each with a reason tied to a hypothesis or to the modeling problem, not added just to have more columns. The `native_region` feature groups native country into US, non-US, and Unknown. The raw column is about 90 percent United States spread across 42 categories, so one-hot encoding it directly would produce dozens of nearly empty columns. Grouping it gives the split my fairness audit needs for Hypothesis 2 while keeping missing-country people separate instead of quietly folding them into one side. The `has_capital_gain` and `has_capital_loss` flags mark whether a person has any capital gain or loss at all. Those two columns are almost always zero with a handful of very large values, so a flag captures the wealth signal without the model over-reacting to the raw dollar amount, and it gives the linear model a threshold effect it could not otherwise express. The `hours_group` feature bins hours per week into ranges, because my EDA showed the effect of hours rises and then flattens above full time, so binning lets the linear model express that shape and makes the education versus hours comparison in Hypothesis 1 fairer to hours instead of handicapping it.

I do not reduce dimensions and I do not drop features by importance filtering. I ran PCA on the numeric features in the EDA milestone and the variance spread fairly evenly across components, so there was no small set that captured almost everything. There are only seven numeric features, the correlation heatmap showed no strong multicollinearity, and Hypothesis 1 depends on knowing which original feature matters, which PCA components would destroy. Reducing here would cost interpretability without buying anything.

The guidelines list a before and after comparison of the feature set as optional, and I ran it because it is cheap and turns an assumption into a measurement. I fit the same default logistic regression twice on the same split, once on the raw columns only and once with the four engineered features added, and compared on the validation set.

Feature set	Columns	Recall	F1	PR-AUC
Raw features only	84	0.849	0.685	0.759
With engineered features	52	0.850	0.689	0.773

The engineered set is both smaller and stronger, 52 columns against 84 and better on every metric, so the four features are doing work and not just adding noise.

7. Train, validation, and test split, and confirming no leakage

I use a three-way stratified split instead of a single train and test split, so I have a dedicated validation set to make every decision on before I touch the test set. Training is 60 percent at 19,521 rows, validation is 20 percent at 6,508 rows, and test is 20 percent at 6,508 rows, stratified on the target so the 0.241 positive rate is preserved in all three. The training set fits the models, the validation set is where I compare untuned versions, tune the hyperparameters, and choose the group thresholds in section 20, and the test set stays untouched until the final evaluation so the last numbers are honest.

Four things confirm no leakage occurred. Scaling and SMOTE both live inside an imbalanced-learn pipeline, so when the pipeline is fit they are fit on training rows only and then applied to the held-out rows without being refit. Because they live in the pipeline, cross-validation refits both inside each fold, so the scaler never sees the validation fold and SMOTE never generates a synthetic row from one. SMOTE is applied after the split and never before, so no synthetic row derived from a training person can appear in validation or test. And the test set is used in exactly one place, the final evaluation, with every tuning decision made on validation.

8. Methodology: the experimental pipeline

The pipeline runs start to finish as follows. Load the raw CSV into SQLite once, then pull every row back out with a SQL query. Clean: fill the three categorical columns that have gaps with Unknown, drop the 24 exact duplicates before dropping the sampling weight, drop the sampling weight and the redundant text education column, and build the 0/1 target. Engineer: add native_region, the two capital flags, and hours_group. Encode: one-hot the eight categorical columns with the first level dropped, giving a 52 column model matrix. Split: a stratified 60/20/20 train, validation, test split with a fixed seed. Fit: inside one imbalanced-learn pipeline, standardize the five continuous columns, apply SMOTE to the training rows, then fit the classifier. Tune: fit each hyperparameter setting on the training set and score PR-AUC on the validation set, then keep the best. Cross-validate: 5-fold stratified cross-validation on the training rows, with identical folds across all three models so the fold scores are paired. Evaluate: score every model on the untouched test set, then run paired statistical tests and bootstrap confidence intervals on the differences. Audit: split the test rows by nativity and by sex, compute recall inside each group for each model, and test whether the gaps differ between models.

The cross-validation is 5-fold and stratified on the training rows. Five folds keep enough minority-class rows in each fold for a steady PR-AUC, and stratifying holds the 24 percent positive rate in every fold. The important detail is that I generate the fold indices once and reuse the same ones for all three models. That is what makes the fold scores paired, and paired scores are the only thing that lets me run a paired t-test on them with five observations. One seed, 42, is used on the split, the folds, SMOTE, and all three models.

Each of the three pipelines builds its own scaler instead of sharing one object. A scikit-learn pipeline does not copy the steps it is handed, so when all three models pointed at a single ColumnTransformer, refitting one model later silently changed the preprocessing of the models I had already scored. XGBoost's test PR-AUC read 0.744

instead of 0.805 and nothing raised an error. Building a fresh scaler per pipeline fixed it, and I mention it here because it is a reproducibility property of the code, not a detail of the analysis.

The software is Python with numpy, pandas, scikit-learn, imbalanced-learn, xgboost, scipy, matplotlib, and seaborn. Exact versions are in section 27.

9. The models, and how each one makes a prediction

The naive baseline is a majority-class predictor that labels everyone as under 50K regardless of their features. It exists to make the accuracy trap concrete: it scores about 76 percent accuracy while catching zero real high earners, so any model that cannot beat it on recall and PR-AUC has learned nothing.

Logistic regression is my interpretable baseline. It takes a weighted sum of the features, passes that through the sigmoid to turn it into a probability, and applies a 0.5 threshold for a yes or no. The decision function and the sigmoid are:

$$z = b_0 + \sum_j b_j x_j, \quad p = \text{sigma}(z) = 1 / (1 + e^{-z})$$

where x_j is the value of feature j for a person, b_j is the coefficient learned for that feature, b_0 is the intercept, and p is the predicted probability that the person earns over 50K. It is fit by minimizing the log loss, which penalizes confident wrong answers far more than uncertain ones:

$$L = -(1/n) \sum_i [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] + (1/C) R(b)$$

where n is the number of training rows, y_i is the true 0/1 label for row i , p_i is the predicted probability for that row, $R(b)$ is the regularization penalty on the coefficients, and C is the inverse regularization strength. The limit that matters is that it is linear and additive, so it misses interactions and curves unless I hand-engineer them, and that limit is exactly what Hypothesis 3 tests.

XGBoost builds shallow decision trees one at a time, where each new tree is fit to the errors the ensemble has made so far, and the prediction is the sum of every tree's output passed through the sigmoid:

$$y = \text{sigma}(\sum_k \eta f_k(x))$$

where f_k is the k -th tree, $f_k(x)$ is the value that tree assigns to a person by dropping them down its splits, and η is the learning rate that shrinks each tree's contribution. Each tree is grown by choosing splits that most reduce a regularized objective, so it adds a tree only where the gain is worth the extra complexity:

$$\text{Objective} = \sum_i \text{loss}(y_i, \hat{y}_i) + \sum_k (\gamma T_k + (1/2) \lambda \|w_k\|^2)$$

where T_k is the number of leaves in tree k , w_k are that tree's leaf values, γ is the cost of adding a leaf, and λ is the L2 penalty on the leaf values. Because a split can happen at any value and later trees can split again inside a region an earlier tree carved out, it captures thresholds, curves, and interactions without being told they exist. That is the mechanism behind my Hypothesis 3 expectation.

The multilayer perceptron passes the features through one or more hidden layers of neurons, where each neuron takes a weighted sum of its inputs and applies a non-linear activation, and the output layer turns the final values into a probability. For a single hidden layer:

$$h = \text{ReLU}(W_1 x + b_1), \quad p = \text{sigma}(W_2 h + b_2)$$

where x is the feature vector for one person, $W1$ and $b1$ are the weights and biases of the hidden layer, h is the hidden representation, $\text{ReLU}(u)$ is the maximum of 0 and u , and $W2$ and $b2$ map that representation to the output. Stacking non-linear layers lets it approximate interactions too, but it has to learn them from the data with no structural head start, which is why I expect it to need more rows than this dataset has.

10. Evaluation metrics and why each was chosen

Writing TP, TN, FP, and FN for true and false positives and negatives:

$$\begin{aligned} \text{Accuracy} &= (TP + TN) / (TP + TN + FP + FN) & \text{Precision} &= TP / (TP + FP) & \text{Recall} &= TP / (TP + FN) \\ F1 &= 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \end{aligned}$$

Accuracy is the overall share correct. I report it but never lead with it, because the majority-class floor already scores about 76 percent. Precision is the share of the people I flag as high earners who really are, which for the lender is wasted review effort. Recall is the share of the real high earners I actually catch, which for the lender is lost business and for the applicant is a denied opportunity, and it is the metric the whole fairness audit is built on, because a recall gap means the model finds one group's high earners and misses the other's. F1 is the harmonic mean of precision and recall, so it stays low unless both are decent, which makes it a fair single summary of threshold performance.

PR-AUC, the area under the precision-recall curve, is my lead metric. It focuses on the positive class directly, does not get credit for correctly rejecting the easy majority, and has a no-skill floor at the 0.241 positive rate instead of 0.5. It is also threshold-free, so it compares how well the models rank people rather than where their cutoff happens to land. ROC-AUC stays as a secondary number because it flatters on imbalanced data, where a large easy negative class props the curve up.

11. Statistical comparison methods and their assumptions

This is the part that makes the model comparison mean anything. Reporting that one model's PR-AUC is higher than another's is not a finding on its own, because fold-to-fold variation could produce that ordering by chance. I use four methods, each answering a slightly different question.

The paired t-test runs on the cross-validation fold scores. The three models are scored on the same five folds, so each fold gives one score per model and those scores are paired, not independent. The test asks whether the mean of the per-fold differences is far enough from zero relative to how much those differences vary. Pairing removes the fold-to-fold difficulty that affects all three models equally, which is the entire reason it has any power with only five folds. It assumes the per-fold differences are roughly normal and that the folds are independent of each other. With five folds normality cannot be checked seriously, which is why I run a second test that does not need it.

The Wilcoxon signed-rank test runs on the same fold differences. It keeps only the signs and ranks of the differences, so it drops the normality assumption entirely, and it assumes only that the differences are symmetric. If the two tests agree then the conclusion does not depend on the normality assumption. With five folds it is underpowered, so I read it as a check and not as the headline, for a reason I explain in section 15.

McNemar's test runs on the held-out test predictions. The three models predict on the same 6,508 test rows, so their errors are paired at the row level. McNemar's test looks only at the rows where two models disagree and asks whether the split of those disagreements is far from 50/50. Its only assumption is that the paired disagreements are

independent across rows. This is the right test for comparing two classifiers on one test set, and I use the exact binomial version instead of the chi-square approximation so it stays valid even when there are few discordant pairs.

The paired bootstrap gives confidence intervals on the test set. I resample the test rows with replacement 1,000 times and recompute the metric for every model on the same resample each time. Because every model sees the identical resample, the difference between two models can be computed inside each replicate, which gives a confidence interval directly on the difference instead of two separate intervals I would have to eyeball for overlap. That distinction matters, because overlapping intervals do not actually mean a difference is non-significant. The bootstrap needs no distributional assumption at all, which is why it is the tool I use for PR-AUC and for the fairness gaps, where the sampling distribution has no clean closed form.

12. The naive baseline and the untuned models

Before any real model I fit the majority-class predictor and check it on the validation set. It scores 0.759 accuracy while catching zero real high earners, and its PR-AUC sits at the 0.241 base rate, which is the floor PR-AUC can reach, so there is no signal at all. A useless model already scores in the high 70s on accuracy, and the recall of zero is what gives it away. That is why I lead with PR-AUC and recall.

Every model then gets fit once with library defaults, which is the before state I compare the tuned version against later. The untuned logistic regression with SMOTE is my true baseline, the one my professor's checklist asks for, and it is what the other two have to beat. I also fit the logistic regression on the raw imbalance so the effect of SMOTE on its own is still visible.

Model	Accuracy	Precision	Recall	F1	PR-AUC	ROC-AUC
Naive (majority class)	0.759	0.000	0.000	0.000	0.241	0.500
LR, raw imbalance	0.848	0.722	0.603	0.657	0.771	0.909
LR with SMOTE (the baseline)	0.815	0.580	0.850	0.689	0.773	0.909
XGBoost	0.856	0.690	0.730	0.709	0.809	0.922
MLP	0.819	0.594	0.793	0.679	0.744	0.893

Adding SMOTE to the logistic regression trades precision for a large jump in recall, from 0.603 to 0.850, which is the trade I want for a lender that would rather review a borderline case than miss a real high earner. Accuracy actually falls when I add SMOTE, from 0.848 to 0.815, and that is the point: if I were optimizing accuracy I would have thrown SMOTE out and built a worse model for this use case. Out of the box XGBoost already leads on PR-AUC at 0.809, and the untuned MLP lands below the logistic baseline at 0.744, which is the first sign that this dataset is not large enough for the neural network to pull ahead.

13. Hyperparameter tuning

I tune every model the same way: fit each setting on the training set, score PR-AUC on the validation set, and keep the best. The test set is not involved. PR-AUC is the search criterion because it is my lead metric and because it is threshold-free, so the search optimizes how well the model ranks people instead of where its 0.5 cutoff lands. I use grid search rather than random search because each model has only two or three parameters worth moving here, so the grid is small enough to cover exhaustively and I would rather know I checked every combination.

13.1 What each parameter controls and why it affects performance

For logistic regression, `C` is the inverse regularization strength. A small `C` means stronger regularization, which shrinks the coefficients toward zero and gives a simpler model that risks underfitting, while a large `C` means weaker regularization, so the model fits the training data more closely and risks overfitting. This is the bias-variance dial for this model. The penalty is the shape of that regularization: L2 shrinks all coefficients smoothly and keeps every feature, while L1 can drive some coefficients to exactly zero and act like a light feature selector, which can help when many of the 52 one-hot columns carry little signal.

For XGBoost, `max_depth` is how deep each tree can grow. Deeper trees can express interactions between more features at once but memorize noise, so this is the main overfitting dial for a boosted ensemble. The `learning_rate` is how much each new tree contributes: a smaller rate makes each correction more cautious, which usually generalizes better but needs more trees to reach the same fit, so it trades against `n_estimators` directly. The `n_estimators` parameter is how many trees get built, where too few underfits because the ensemble has not finished correcting itself and too many overfits because later trees start fitting noise in the residuals. The `subsample` parameter is the fraction of rows each tree sees, and below one it injects randomness that decorrelates the trees, reducing variance in the same way bagging does.

For the MLP, `hidden_layer_sizes` is the shape and capacity of the network. More or wider layers can represent more complex functions but need more data to fit them, which is the constraint I flagged for Hypothesis 3. The `alpha` parameter is the L2 penalty on the weights, the same bias-variance dial as `C` on the logistic regression but pointed the other way round, since a larger `alpha` means more regularization. The `learning_rate_init` parameter is the size of the optimizer's steps: too large and it bounces past good solutions, too small and it stops early on a plateau before it has fit anything.

13.2 The exact search space

The logistic regression search is 8 combinations: `C` in 0.01, 0.1, 1, 10 crossed with penalty in L1 and L2, using the liblinear solver since it is the one that supports both penalties. The XGBoost search is 12 combinations: `max_depth` in 3, 5, 7 crossed with `learning_rate` in 0.05 and 0.1 crossed with `n_estimators` in 200 and 400, with `subsample` and `colsample_bytree` held at 0.8 instead of searched, because I wanted the randomness they add present in every candidate instead of turning it into another dimension. The MLP search is 6 combinations: `hidden_layer_sizes` in (50,), (100,), and (64, 32) crossed with `alpha` in 0.0001 and 0.01, with `learning_rate_init` held at 0.001, early stopping on, and `max_iter` at 400, because early stopping already guards against the failure mode a bad learning rate would cause and I would rather spend the search on capacity. That is 26 fits in total, every one fit on the 19,521 training rows and scored on the 6,508 validation rows.

13.3 Final chosen values, tuned against untuned

Model	Chosen parameters	Untuned val PR-AUC	Tuned val PR-AUC	Change
Logistic regression	<code>C</code> = 10, penalty = L1	0.7728	0.7737	+0.0009
XGBoost	<code>max_depth</code> = 7, <code>learning_rate</code> = 0.05, <code>n_estimators</code> = 400, <code>subsample</code> = 0.8	0.8091	0.8110	+0.0019
MLP	<code>hidden_layer_sizes</code> = (50,), <code>alpha</code> = 0.01	0.7435	0.7747	+0.0311

Model	Accuracy	Precision	Recall	F1	PR-AUC	ROC-AUC
LR untuned	0.815	0.580	0.850	0.689	0.773	0.909
LR tuned	0.815	0.580	0.849	0.689	0.774	0.909
XGBoost untuned	0.856	0.690	0.730	0.709	0.809	0.922
XGBoost tuned	0.858	0.689	0.747	0.717	0.811	0.924
MLP untuned	0.819	0.594	0.793	0.679	0.744	0.893
MLP tuned	0.830	0.610	0.816	0.698	0.775	0.910

Tuning moves the three models by very different amounts, and the pattern tells me something useful. The logistic regression gains 0.0009 because it is a low variance model with only two knobs, and the validation curve I ran in the baseline milestone was already flat by about $C = 1$, so it was near its ceiling before I started. XGBoost gains 0.0019 because its defaults are already close to sensible for tabular data of this size, and the search mostly confirmed that a moderately deep tree with a slower learning rate and more trees is the right region. The MLP gains 0.0311, which makes sense because it has the most capacity and the least structural prior. The smallest network with the strongest weight penalty won and the two-layer network came last. A model that gets better when I make it smaller and regularize it harder is telling me it does not have enough data for its capacity, which is the same conclusion the learning curves reach in section 23.

14. Cross-validation on identical folds

A single validation number can be lucky, so I run 5-fold stratified cross-validation on the training rows, with the same five folds handed to all three models.

Model	PR-AUC mean	PR-AUC std	PR-AUC var	F1 mean	F1 std	Recall mean	Accuracy mean
Logistic regression	0.7797	0.0105	0.0001	0.6876	0.0109	0.8371	0.8168
MLP	0.7878	0.0143	0.0002	0.6905	0.0102	0.8348	0.8197
XGBoost	0.8186	0.0114	0.0001	0.7198	0.0101	0.7385	0.8615

Every standard deviation is around 0.01 and every variance is one to two ten-thousandths, so no model is riding on one lucky split. That ratio matters for the next section: the fold-to-fold noise is small relative to the 0.039 gap between XGBoost and the baseline, which is why the tests have any chance of detecting it. If the noise had been the same size as the difference, no test would have found anything and I would have had to say so. XGBoost leads on PR-AUC, F1, and accuracy but has the lowest recall, because it sits at a more conservative operating point, and that fact comes back in the fairness audit.

15. Statistical tests on the model comparison

Metric	Comparison	Mean diff	t	t p-value	Wilcoxon p	Significant at 0.05
PR-AUC	XGBoost vs LR	0.0389	12.99	0.0002	0.0625	yes
PR-AUC	XGBoost vs MLP	0.0308	7.72	0.0015	0.0625	yes
PR-AUC	MLP vs LR	0.0081	4.34	0.0123	0.0625	yes
F1	XGBoost vs LR	0.0322	7.12	0.0021	0.0625	yes
F1	XGBoost vs MLP	0.0293	10.51	0.0005	0.0625	yes
F1	MLP vs LR	0.0028	1.41	0.2317	0.1875	no

On PR-AUC every comparison is significant by the paired t-test. On F1 the two XGBoost comparisons are significant and the MLP against the logistic regression is not, at $p = 0.23$, so the MLP has a small ranking advantage over the baseline that disappears once a threshold is applied.

Every Wilcoxon p-value on the significant comparisons comes back at exactly 0.0625, and that number is not a coincidence I should read as weak evidence. With five paired observations the smallest two-sided p-value the Wilcoxon signed-rank test can produce is 0.0625, because there are only 32 possible sign patterns and the most extreme one still leaves one sixteenth of the probability in the two tails. The test literally cannot return anything under 0.05 at this sample size no matter how clean the result is. That is a limitation of running a rank test on five folds, not evidence against the difference, and the 0.0625 does tell me something useful, which is that XGBoost won on all five folds with no exceptions. The tests with real sample size behind them are McNemar and the bootstrap, and both use all 6,508 test rows.

16. Confidence intervals, and what their width means

The 95 percent confidence intervals on cross-validated PR-AUC, computed as the mean plus or minus $t(0.975, 4)$ times the standard error of the fold scores, are 0.7667 to 0.7927 for the logistic regression, 0.7701 to 0.8055 for the MLP, and 0.8044 to 0.8328 for XGBoost.

A confidence level and an interval width trade against each other, and the clearest way to show it is on my own numbers. Taking the XGBoost minus logistic regression PR-AUC difference across the same five folds:

Confidence level	Interval	Width	Excludes zero
90 percent	0.0325 to 0.0453	0.0128	yes
95 percent	0.0306 to 0.0472	0.0166	yes
99 percent	0.0251 to 0.0527	0.0276	yes

The 90 percent interval is the narrowest and the 99 percent interval is the widest, because demanding more certainty that the interval covers the true difference means casting a wider net, and nothing about the data changed between those three rows. So a narrow interval is only worth something if it is narrow at a high confidence level. Reporting a tight 90 percent interval and calling the difference settled would be cheating, because I would have bought that tightness by lowering the bar. The only real way to get a narrow interval at a high confidence level is more data, since the width shrinks with the square root of the sample size. That is why I do not rely on five folds alone, and it works in my favor here: the bootstrap interval on the same difference uses all 6,508 test rows and comes out 0.0184 wide at 95 percent, narrower than any of the three above. In section 19 the same principle works against me.

17. Final evaluation on the held-out test set

I touch the test set here for the first time. Every choice above was made on the training and validation sets. The table reports all three tuned models against the naive floor and against the untuned baseline, which is the comparison the guidelines ask for: a naive baseline, my untuned baseline model, and alternative model types.

Model	Accuracy	Precision	Recall	F1	PR-AUC	ROC-AUC
Naive (majority class)	0.759	0.000	0.000	0.000	0.241	0.500
LR untuned (the baseline)	0.816	0.585	0.818	0.682	0.773	0.904
LR tuned	0.818	0.587	0.820	0.684	0.774	0.904
MLP tuned	0.825	0.604	0.789	0.684	0.775	0.903
XGBoost tuned	0.857	0.695	0.721	0.708	0.805	0.915

The test set confirms the validation ordering, which is what I want to see, since a model that led on validation and then fell apart on test would mean I had overfit my tuning to the validation rows. Every model clears the naive floor by a wide margin, and the naive model's 0.759 accuracy is higher than what the logistic regression manages, which is the clearest possible justification for not reporting accuracy as a headline metric. One thing to notice before the fairness section is that XGBoost has the highest F1 but the lowest recall of the three, 0.721 against 0.820 for the logistic regression. It is trading caught high earners for far fewer false alarms, 495 against 905, so the three models are not sitting at the same operating point.

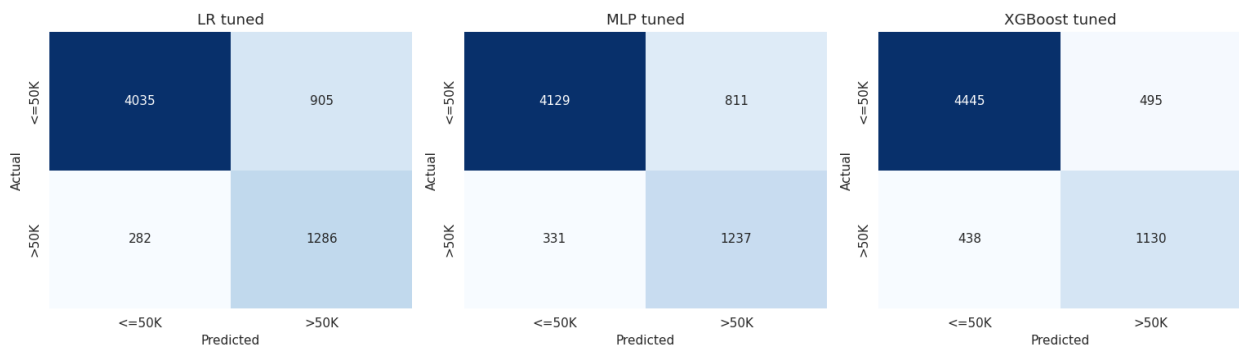


Figure 2. Confusion matrices on the same test rows. The logistic regression catches the most real high earners in the bottom right cell but pays with 905 false alarms in the top right. XGBoost moves many of those back to the correct cell while giving up some caught high earners, which is where its higher F1 comes from.

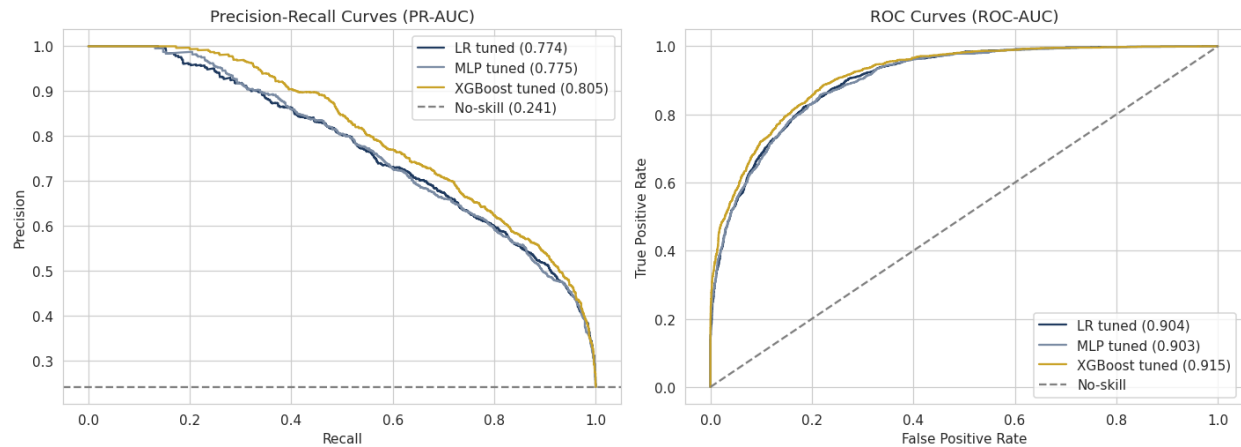


Figure 3. Precision-recall and ROC curves. XGBoost sits above the other two across nearly the whole recall range, so at any recall level it flags fewer people wrongly, and all three stay well above the 0.241 no-skill line. The ROC curves are much closer together, which is the flattering behavior on imbalanced data and the reason PR-AUC is my lead metric instead of ROC-AUC.

17.1 Bootstrap confidence intervals on the test results

Model	Metric	Estimate	95 percent CI	CI width
LR tuned	PR-AUC	0.7743	0.7549 to 0.7920	0.0371
MLP tuned	PR-AUC	0.7750	0.7563 to 0.7931	0.0368
XGBoost tuned	PR-AUC	0.8052	0.7878 to 0.8217	0.0338
LR tuned	F1	0.6838	0.6668 to 0.7003	0.0334
MLP tuned	F1	0.6836	0.6652 to 0.7000	0.0349
XGBoost tuned	F1	0.7076	0.6888 to 0.7250	0.0362

Metric	Comparison	Mean difference	95 percent CI	Excludes zero	Win rate
PR-AUC	XGBoost minus LR	0.0309	0.0218 to 0.0402	yes	100 percent
PR-AUC	XGBoost minus MLP	0.0302	0.0221 to 0.0378	yes	100 percent
PR-AUC	MLP minus LR	0.0007	-0.0071 to 0.0090	no	58 percent
F1	XGBoost minus LR	0.0238	0.0101 to 0.0368	yes	100 percent
F1	XGBoost minus MLP	0.0240	0.0123 to 0.0367	yes	100 percent
F1	MLP minus LR	-0.0002	-0.0103 to 0.0105	no	48 percent

17.2 McNemar's exact test on the shared test rows

Comparison	Rows only the first got right	Rows only the second got right	Discordant pairs	Exact p-value
XGBoost vs LR	454	200	654	1.2e-23
XGBoost vs MLP	373	164	537	9.6e-20
MLP vs LR	272	227	499	0.049

For XGBoost against either of the other two, four independent readings point the same way: the paired t-test is significant, the bootstrap interval on the difference excludes zero comfortably, XGBoost won in 1,000 of 1,000 bootstrap resamples, and McNemar returns a p-value far below any threshold. That is as settled as this dataset can make it. The MLP against the logistic regression is less clear: the fold t-test says the MLP is ahead on PR-AUC at $p = 0.012$, McNemar comes back at $p = 0.049$ which is barely under the line, and the bootstrap interval on the PR-AUC difference runs from -0.007 to 0.009 and contains zero. When tests disagree I go with the one that has the most data behind it and the fewest assumptions, which is the bootstrap, so my reading is that the MLP and the logistic regression are not meaningfully different on this dataset. The practical statement is that the extra complexity of the MLP bought nothing over a linear model here, while XGBoost bought a real improvement.

18. Does the model beat the baseline, and is the difference significant

The guidelines ask me to state plainly whether my model beats the baseline and whether that difference is statistically significant, and to treat those as two separate claims that each need their own evidence. So here they are, one at a time.

The first claim is that XGBoost beats the baseline. It does. On the held-out test set XGBoost reaches 0.805 PR-AUC against 0.773 for the untuned logistic regression baseline and 0.774 for the tuned one, and 0.708 F1 against 0.682 and 0.684. It also beats the naive majority-class floor, which sits at 0.241 PR-AUC and catches zero real high earners. That is a claim about the point estimates only. It says the numbers came out higher and nothing more.

The second claim is that the difference is statistically significant. It is, and it needs its own evidence, because a higher number could come from which rows happened to land in the test set. The paired t-test on five shared folds gives $p = 0.0002$ on PR-AUC. McNemar's exact test on the 6,508 shared test rows gives p on the order of $1e-23$. The paired bootstrap interval on the PR-AUC difference is 0.0218 to 0.0402, which excludes zero, and XGBoost won in 1,000 of 1,000 resamples. Both claims hold for XGBoost against the baseline.

The MLP is the case where the two claims come apart, which is exactly why separating them matters. The first claim is arguably yes, since its test PR-AUC of 0.775 is a hair above the baseline's 0.773. The second claim is no: the paired bootstrap interval on that difference runs from -0.0071 to 0.0090 and contains zero. A model can post a higher number and still have no demonstrated advantage, and reporting only the first claim would have hidden that.

19. Fairness audit: performance split by group

This is where Hypothesis 2 gets tested, and it is also where I deal with the two group imbalances in this data. About 90 percent of the people are US-born and about 67 percent are men, so both minority groups are seen far less often by every model during training. The metric I audit on is recall inside each group on real high earners, because that is the error that hurts the person and not the lender. A missed high earner is somebody who qualifies and does not get flagged, and if the model misses more of them in one group than another, the model works worse for that group.

Grouping	Group	Test rows	Share of test set	True over 50K rate	Real high earners
Nativity	US-born	5,819	89.4 percent	0.247	1,436
Nativity	Foreign-born	576	8.9 percent	0.172	99
Sex	Male	4,335	66.6 percent	0.306	1,327
Sex	Female	2,173	33.4 percent	0.111	241

I report accuracy by group as well, because that is the plain version of the same question, but I do not lead with it. Every model is more accurate inside the foreign-born group than the US-born group and far more accurate on women than on men, which would read as the model working better for those groups if I stopped there. It is not working better. Those groups simply have fewer high earners to find, so predicting under 50K is right more often. Recall on real high earners is the metric that shows what is actually happening.

Model	Recall US-born	Accuracy US-born	Recall foreign-born	Accuracy foreign-born	Recall gap
LR tuned	0.823	0.816	0.768	0.844	0.055
MLP tuned	0.793	0.823	0.747	0.851	0.046
XGBoost tuned	0.726	0.855	0.646	0.882	0.079

Model	Recall male	Accuracy male	Recall female	Accuracy female	Recall gap
LR tuned	0.836	0.766	0.730	0.920	0.106
MLP tuned	0.806	0.775	0.697	0.924	0.108
XGBoost tuned	0.743	0.818	0.598	0.935	0.146

The point estimates say the opposite of what Hypothesis 2 predicted. XGBoost has the largest nativity recall gap of the three at 0.079, against 0.055 for the logistic regression and 0.046 for the MLP, and the same ordering shows up on sex.

19.1 Confidence intervals on the gaps

Gap	Model	Estimate	95 percent CI	CI width	Gap is real
US minus non-US	LR tuned	0.034	-0.037 to 0.111	0.1479	no
US minus non-US	MLP tuned	0.050	-0.023 to 0.129	0.1520	no
US minus non-US	XGBoost tuned	0.059	-0.025 to 0.146	0.1707	no
Male minus female	LR tuned	0.106	0.050 to 0.163	0.1125	yes
Male minus female	MLP tuned	0.109	0.050 to 0.171	0.1212	yes
Male minus female	XGBoost tuned	0.146	0.081 to 0.213	0.1321	yes

Every nativity gap interval contains zero, including XGBoost's, so I cannot claim any of the three models has a nativity gap that is distinguishable from no gap at all on this test set. The sex gaps are a different story: all three intervals exclude zero, so those gaps are real, and every model finds male high earners more reliably than female high earners by about 11 to 15 percentage points. The comparison between models is not settled on either axis,

because no interval on the difference between two models' gaps excludes zero. The direction leans against Hypothesis 2, with XGBoost's sex gap smaller than the logistic regression's in only 6.9 percent of resamples and its nativity gap smaller in only 27.5 percent, but leaning is not the same as showing.

19.2 How much more data would settle it

This is what the point about interval width looks like in practice and sample size from section 16, and here it works against me. The nativity gap is estimated from only 99 real foreign-born high earners in the test set, and with that little to work with the 95 percent interval spans about 0.171, which is wider than any gap I could plausibly find. The interval is not wide because the models are unstable, it is wide because the group is small.

Target CI width	Sample size multiple needed	Foreign-born high earners needed
0.05	11.6 times	1,153
0.02	72.8 times	7,208

So the right conclusion is not that XGBoost and the logistic regression have equal nativity gaps. It is that this dataset does not contain enough foreign-born high earners to tell them apart, and reporting a point estimate without that caveat would be the mistake. The sex comparison has 241 female high earners instead of 99, and that is already enough for the gaps themselves to come out significant even though the model-to-model differences still do not. The cross-validation folds, which use different rows, give a different ordering again, with the MLP showing the largest mean nativity gap at 0.091. Two ways of splitting the same data disagreeing about the ordering is exactly what an underpowered comparison looks like.

20. Handling the group imbalance

Finding a gap is only half the job. The other half is trying to close it and measuring what that costs. I test three approaches against the as-is model, for both nativity and sex. The first is training on the minority group only, since every model above is trained on data where one group dominates, so the obvious question is whether the gap comes from the model simply not seeing enough of the smaller group. The second is group-balanced oversampling, where I upsample the minority group's training rows with replacement until both groups are equally represented and then train normally, which keeps all the majority data while removing the representation imbalance. The third is a group-specific decision threshold, where I pick a separate cutoff for each group on the validation set so both groups reach a similar recall, then apply those cutoffs to the test set. Thresholds are chosen on validation only, never on test.

Model	Strategy	Recall US-born	Recall foreign-born	Gap	Overall F1
LR	as-is (full data)	0.823	0.768	0.055	0.684
LR	minority group only	0.930	0.768	0.162	0.617
LR	group-balanced oversample	0.838	0.778	0.060	0.669
LR	group thresholds	0.779	0.798	-0.019	0.675
XGBoost	as-is (full data)	0.726	0.646	0.079	0.708
XGBoost	minority group only	0.634	0.667	-0.032	0.653
XGBoost	group-balanced oversample	0.742	0.717	0.024	0.706

Model	Strategy	Recall US-born	Recall foreign-born	Gap	Overall F1
XGBoost	group thresholds	0.767	0.778	-0.011	0.696
MLP	as-is (full data)	0.793	0.747	0.046	0.684
MLP	minority group only	0.724	0.778	-0.054	0.660
MLP	group-balanced oversample	0.788	0.697	0.091	0.661
MLP	group thresholds	0.768	0.768	0.000	0.680

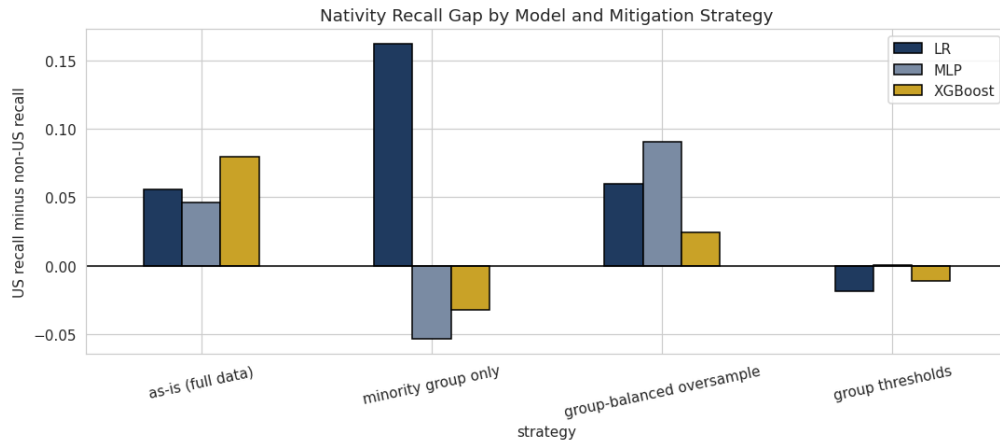


Figure 4. The nativity recall gap for every model and mitigation strategy. Lower is fairer and zero is parity. Group thresholds bring all three models close to parity, and group-balanced oversampling gets most of the way there for XGBoost at almost no cost to F1.

Training on the minority group only is the worst of the four, and it fails in a way that tells me something. The foreign-born model has 1,680 training rows against 19,521, so it is starved, and the female-only XGBoost model collapses to 0.375 recall on men and 0.549 overall F1. Cutting the data down to the group did not fix the gap, it just moved which group gets served badly. That answers the question the experiment was designed to ask: the gap is not simply a matter of the model never seeing enough foreign-born or female rows, because giving it nothing but those rows makes things worse, not better.

Group-balanced oversampling is the mild version of the same idea and it works better because it keeps everything. For XGBoost it cuts the nativity gap from 0.079 to 0.024 and the sex gap from 0.146 to 0.099, and it costs almost nothing on overall F1, 0.708 down to 0.706 and 0.703. Group-specific thresholds are the most effective by a distance, closing the nativity gap to within about one point of zero for all three models and turning the sex gaps slightly negative, at a cost of 0.708 down to 0.696 F1 for XGBoost. That works because the imbalance is not really about what the model learned, it is about where a single 0.5 cutoff falls on two different score distributions, and moving the cutoff per group is the direct fix.

The important caveat is that a group-specific threshold means explicitly using nativity or sex to decide somebody's outcome, which in a real lending setting is very likely illegal under fair lending rules regardless of the intent. So the honest recommendation is group-balanced oversampling, which fixes a good share of the gap without the model ever branching on the protected attribute at prediction time, and the thresholds are best read as a diagnostic. That diagnostic changes how the Hypothesis 2 result should be read, because once all three models are put at a matched recall of about 0.78 by the group thresholds, XGBoost's nativity gap is -0.011 and it still has the highest F1 of the three. Its larger raw gap is at least partly a consequence of it operating at a much lower recall

than the other two, not evidence that it treats the groups more differently.

21. Testing the three hypotheses

21.1 Hypothesis 1 is supported

The test is permutation importance measured on the test set with PR-AUC as the score. I shuffle a feature's values across rows, which breaks its relationship with the target while leaving everything else intact, and measure how far PR-AUC falls. One detail matters for fairness between the two features. Education lives in a single column, `education_num`, but hours lives in `hours_per_week` plus the six `hours_group` dummy columns I engineered. Shuffling only `hours_per_week` would leave the `hours_group` columns telling the model the answer, which would make hours look weaker than it is, so I shuffle the whole hours block together with one shared permutation order. I repeat the shuffle 20 times per feature and use the spread across repeats to build a confidence interval on the difference.

Model	Education drop	Hours drop	Difference	95 percent CI	Education wins
Logistic regression	0.0729	0.0288	0.0441	0.0415 to 0.0467	yes
XGBoost	0.0588	0.0196	0.0392	0.0374 to 0.0411	yes
MLP	0.0692	0.0251	0.0441	0.0415 to 0.0467	yes

Education is worth roughly two and a half to three times as much as hours to every model, and all three intervals exclude zero by a wide margin, so this is not a case where a taller bar could be noise. The agreement across models is what makes it convincing instead of a quirk of one algorithm. This is also the fourth independent method in this project to give the same answer, after the raw rate spread, the decision tree importance, and the Cohen's d effect sizes from the EDA milestone, where education came out as a large effect at 0.83 and hours as a medium one at 0.55.

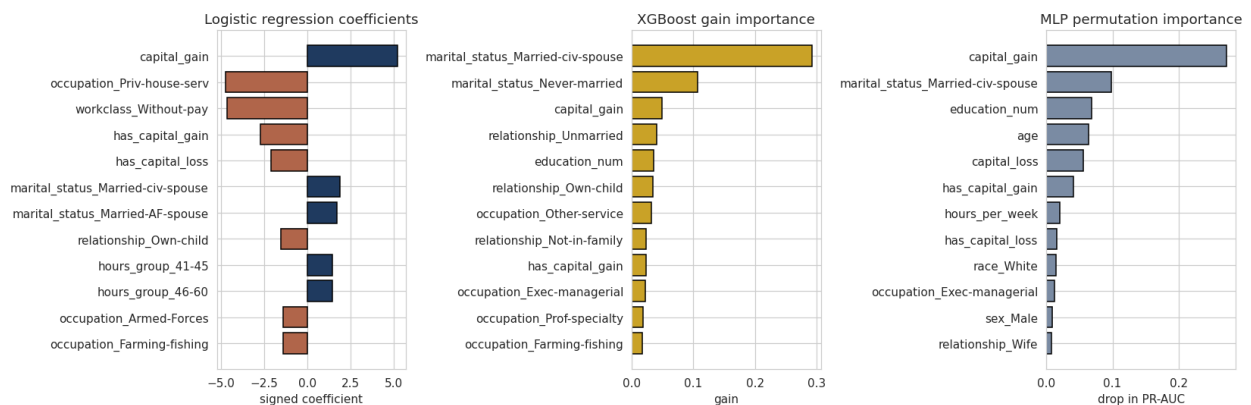


Figure 5. What each model considers important, using the measure native to it. The three panels are not on the same scale, so what I read is the ordering inside each one. Capital gain and marital status dominate all three, and education sits well above hours in every panel.

What Hypothesis 1 does not claim is that education is the single most important feature. Capital gain and marital status outrank it on every model. Hypothesis 1 is a claim about education against hours specifically, and that is the claim the test supports.

21.2 Hypothesis 2 is refuted

XGBoost has the largest raw nativity recall gap of the three at 0.079, not the smallest, against 0.055 for the logistic regression and 0.046 for the MLP, and the same ordering appears on sex. The hypothesis predicted the opposite direction, so on the point estimates it is refuted.

The stronger and more careful statement is that no model is statistically distinguishable from any other on this gap. Every bootstrap interval on a difference between two models' gaps contains zero, and the paired fold tests on the gap return p-values from 0.13 to 0.99. So Hypothesis 2 is not supported, and the reverse hypothesis is not supported either, because 99 foreign-born high earners cannot resolve a 0.03 difference. There is also a confound I have to name instead of hide: the three models sit at very different recall levels, so their gaps are not measured at the same operating point, and at a matched recall XGBoost's nativity gap goes to -0.011. The one thing that is settled here is the sex gap. All three models miss female high earners more often than male ones, by 11 to 15 points, and all three intervals exclude zero, so that gap is real whichever model I pick.

21.3 Hypothesis 3 is supported

On the held-out test set XGBoost reaches 0.805 PR-AUC and 0.708 F1 against 0.774 and 0.684 for the tuned logistic regression and 0.775 and 0.684 for the MLP, and four separate readings agree, as laid out in sections 15, 17, and 18. The reasoning I gave up front is what the result should be read against, and it held on both halves. The non-linear structure was real, so the boosted trees had something to find that the linear model could not express. And the dataset was too small for the neural network, which is visible in the fact that the MLP got better when I shrank it to a single 50-neuron layer and raised its weight penalty, and in its learning curve in section 23.

The part of Hypothesis 3 I would not claim is the MLP beating the logistic regression, since the bootstrap interval on that difference contains zero. The cost side belongs in the verdict too. XGBoost fits in about 3.5 seconds against 2.8 for the logistic regression and 5.3 for the MLP, and all three predict on 6,508 rows in well under a tenth of a second, so compute is not the issue. The real cost is interpretability: the logistic regression hands me one signed coefficient per feature, while XGBoost has 400 trees and no single one carries the answer, so explaining it means gain importance or a shallow surrogate tree, both of which approximate the model instead of describing it. For a lending use case where a declined applicant can ask why, that cost is real.

22. Error analysis

Model	Missed over 50K	Caught over 50K	False alarms	Share of missed with zero capital gain
LR tuned	282	1,286	905	94.7 percent
MLP tuned	331	1,237	811	93.4 percent
XGBoost tuned	438	1,130	495	96.6 percent

The pattern I found in the baseline milestone holds for all three models. The high earners every model misses look ordinary on the strongest features, with about 95 percent of them having zero capital gain against far fewer of the ones each model catches, and they are younger, less educated, and work fewer hours than the ones it finds. XGBoost misses more high earners in total because it sits at a lower recall, but it cuts false alarms almost in half.

The disagreement analysis is where XGBoost's advantage actually lives. On the 654 test rows where XGBoost and the logistic regression disagree, XGBoost is right on 454 and the logistic regression is right on 200. The rows XGBoost fixes have almost twice the average capital gain of the rows it breaks, 460 against 254, which fits the

mechanism: XGBoost can split on capital gain at a threshold and then split again on something else inside that region, while the linear model has to give capital gain one coefficient that applies to everybody.

Subgroup	Rows	True over 50K rate	LR recall	MLP recall	XGBoost recall
Education 9 years or less	2,946	0.133	0.647	0.578	0.488
Education 10 to 12 years	1,931	0.199	0.795	0.761	0.668
Education 13 years or more	1,631	0.486	0.918	0.907	0.861
Age 17 to 30	2,038	0.069	0.603	0.475	0.433
Age 31 to 45	2,493	0.294	0.805	0.803	0.713
Age 46 to 60	1,486	0.389	0.877	0.865	0.824
Age 60 and over	491	0.238	0.897	0.701	0.607

The subgroup table shows every model failing on the same kind of person, not on different people. Recall is worst for people with 9 years of education or less and for people under 30, and best for people with 13 or more years of education, where the true high earner rate is nearly 50 percent. Every model finds high earners easily where they are common and struggles where they are rare, which is the general failure mode, not a quirk of one model. The oldest age band is the exception worth noting: recall drops from 0.897 for the logistic regression to 0.607 for XGBoost among people over 60. That group is only 491 test rows, so it is a place where the flexible model is less reliable than the simple one because it has few examples to work from.

23. Overfitting, underfitting, and generalization

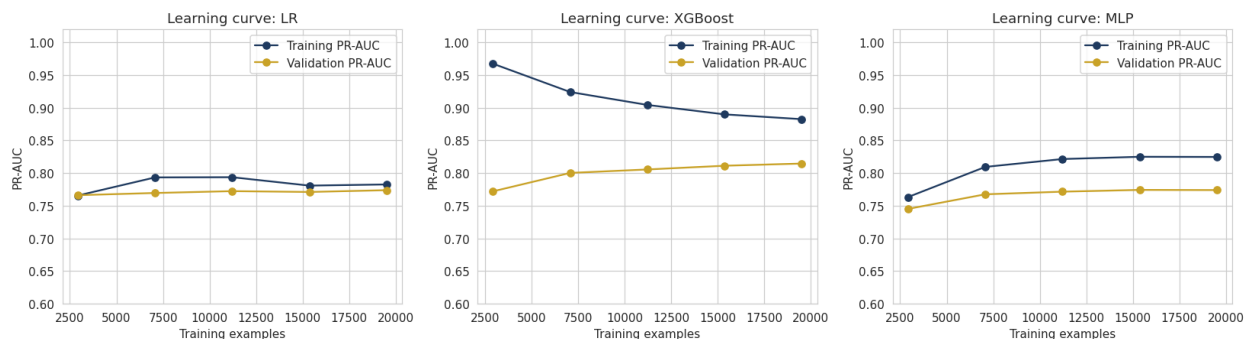


Figure 6. Learning curves for the three models. All three validation lines climb quickly over the first few thousand rows and then flatten, so none of the models is starved of data in the ordinary sense. What separates them is the gap between the training and validation lines.

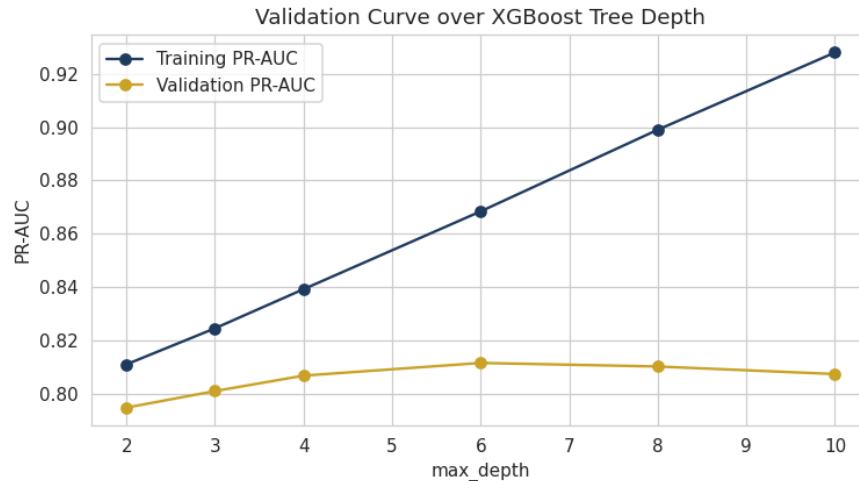


Figure 7. Validation curve over XGBoost tree depth. Training PR-AUC keeps climbing as depth increases while validation peaks and then flattens, which is textbook overfitting past the peak. The tuned depth of 7 sits at that peak.

The learning curves say the same thing for all three models: adding more rows of this kind would not buy much on overall PR-AUC. The gap between the training and validation lines is what separates them. The logistic regression's two lines sit almost on top of each other, which is the signature of a low variance model that is underfitting if anything. XGBoost shows a clear and persistent gap, with training PR-AUC far above validation, which is the memorization a 400-tree, depth-7 ensemble is expected to do. That gap is not a problem in itself, since its validation score is still the highest of the three, but it means the model is using its capacity to fit training rows instead of generalizing further. The MLP's curve has the most movement between sizes, which fits everything else I found about it: it is the model most sensitive to how much data it gets, and 32,500 rows leaves it short. Nothing here suggests XGBoost's advantage is an artifact of overfitting, since the advantage shows up on held-out rows and on every fold.

24. Strengths, limitations, biases, and broader impact

24.1 Strengths

Every comparison in this project is backed by a statistical test with an interval, not by one number being larger than another, and where the tests disagreed, as they did on the MLP against the logistic regression, I said so instead of picking the test that gave the answer I wanted. The cross-validation is properly paired, with all three models seeing the same five folds, so a paired t-test is valid here. The pipeline design makes leakage hard to cause by accident instead of something I had to remember, since the scaler and SMOTE live inside the pipeline and refit inside every fold, and each model builds its own scaler so one model's fit cannot silently change another's. The fairness work goes past reporting a gap: I measured it, put an interval on it, tested whether the models differ, tried three ways of closing it, and reported what each one costs. And the hypotheses were stated with their reasoning before the results, so they could be refuted, and one of them was.

24.2 Limitations

The nativity comparison is underpowered and I could not fix it. With 99 foreign-born high earners in the test set, the interval on that gap is wider than any difference I would want to detect. Grid search on the validation set instead of nested cross-validation means the hyperparameters are tuned on a single split, so while the final numbers are not contaminated, the chosen parameters could be slightly overfit to those 6,508 validation rows. The

three models are compared at the default 0.5 threshold, where they sit at different recall levels, which is a fair comparison of the models as they come out of the box but means the raw group gap comparison is partly about operating points. I did not run calibration curves, which would matter for a real lender, since a probability that is well ranked is not the same as one that is well calibrated. And SMOTE creates synthetic minority rows by interpolating between real ones, which is not obviously meaningful for one-hot encoded categorical columns, where an interpolated value between two categories is not a real category.

24.3 Biases in the data and the model

The data is a 1994 census extract, so it encodes the labor market and the income distribution of one country in one year, and any model trained on it inherits the occupational segregation, education access, and wage gaps of that period. The label itself is a bias: income over 50K was a meaningful threshold in 1994 and is not now, and using recorded income as a proxy for creditworthiness bakes in whatever historical inequity produced that distribution in the first place. The sampling is uneven in exactly the places the audit cares about, with foreign-born people at 9 percent of the data and women at 33 percent, so both groups contribute far less to what every model learns. The model does perform differently across subgroups and I measured it instead of assuming it: recall on female high earners is 11 to 15 points below recall on male high earners for all three models, and that difference is statistically significant, while the nativity difference points the same way but is not measurable at this sample size. Finally, the missing values are not missing at random, since workclass and occupation go missing together, so the Unknown bucket is itself a feature with meaning attached.

24.4 Broader societal impact

If a model like this were deployed as a lending or screening filter, the people affected are applicants who get screened out. A false negative here is a qualified person not being offered something they would have qualified for, and the audit shows those false negatives land more often on women. The obvious misuse is treating a prediction of low income as a reason for an adverse decision: the model has no causal content, it recognizes patterns in a 30-year-old census, so using it to justify a decision about a specific person would be dressing up a correlation as a judgment about them. A second misuse is the fairness fix itself, since the group-specific thresholds close the gap by explicitly using nativity or sex at prediction time, which in a lending context is very likely illegal under fair lending rules no matter how good the intent. The deeper problem is that a model can be fair on one metric and unfair on another at the same time. Every model here looks more accurate on women than on men and misses far more of their high earners, and a report that led with group accuracy would have concluded the opposite of what is actually happening.

24.5 Takeaway lessons

First, a statistical test can be structurally incapable of returning the answer you are looking for. The Wilcoxon test came back at exactly 0.0625 on every clean result because with five folds that is its floor, and reading that as weak evidence would have been a mistake about the test, not about the models.

Second, shared mutable objects will silently corrupt results. All three of my pipelines originally pointed at the same scaler object, and because scikit-learn pipelines do not copy their steps, refitting one model later changed the preprocessing of the models I had already scored. XGBoost's test PR-AUC dropped from 0.805 to 0.744 and nothing raised an error. I only caught it because the test result contradicted the validation result and I went looking for why.

Third, confidence intervals are more useful than p-values when the answer is that you cannot tell. The nativity result is not that the models are equal, it is that 99 people is not enough to distinguish them, and only an interval

says that.

Fourth, the most accurate model is not automatically the fairest one, and on this data it was the least fair on the raw numbers. Building the fairness audit into the comparison instead of adding it at the end is what made that visible.

Fifth, group accuracy and group recall can point in opposite directions on the same model, and on imbalanced groups the one that looks reassuring is usually the wrong one to read.

25. Conclusion

The project asked whether a person earns over 50K can be predicted from 1994 census demographics, and which of three models does it best and most fairly, framed as a lender that flags likely high earners. The three research questions were which feature drives the prediction more, education or hours worked; which model has the smallest US-born versus foreign-born recall gap; and which model classifies best on held-out PR-AUC and F1.

Hypothesis 1 is supported. Education is worth two and a half to three times as much as hours to every model on permutation importance, with confidence intervals on the difference that exclude zero on all three, and it is the fourth independent method in this project to reach that conclusion.

Hypothesis 3 is supported. XGBoost reaches 0.805 PR-AUC and 0.708 F1 on the held-out test set against 0.774 and 0.684 for the tuned logistic regression, it won on all five cross-validation folds and in 1,000 of 1,000 bootstrap resamples, and the paired t-test, McNemar's test, and the bootstrap interval all agree. The part I would not claim is the MLP beating the logistic regression, where the bootstrap interval contains zero.

Hypothesis 2 is refuted. XGBoost has the largest nativity recall gap of the three, not the smallest, and no difference between models is statistically distinguishable, because 99 foreign-born high earners is not enough to resolve a 0.03 difference. What is measurable is that all three models miss female high earners 11 to 15 points more often than male ones, and that gap is significant for every model.

The main finding is the one I did not expect: the most accurate model was not the fairest, and the two questions had to be answered separately, and one did not follow from the other. Group-balanced oversampling recovers most of the fairness gap at almost no cost to F1, which is the configuration I would actually deploy.

For future work, the single change that would do the most for this project is getting more foreign-born rows, or a more recent census extract, so the nativity comparison has the sample size to settle. Beyond that I would compare the models at a matched recall instead of at the default 0.5 threshold so the fairness gaps are measured at the same operating point, add calibration curves and Brier scores since a lender needs probabilities that mean what they say, replace grid search on a single validation split with nested cross-validation so the reported performance accounts for the tuning as well, and try fairness-aware training methods that constrain the gap while fitting instead of correcting it afterward.

26. Reproducibility and references

One fixed seed, 42, is used on the split, the cross-validation folds, SMOTE, and all three models, and there is a single load path from the SQLite database, so the notebook reruns to the same numbers. It runs top to bottom on a clean environment with no manual steps, expecting adult.csv in the working directory and writing adult.db itself. The code is separated so that preprocessing, training, and evaluation are in distinct blocks, and the pipeline factory functions mean every model is built the same way in every place it appears. I ran on numpy 2.4.4, pandas

3.0.2, scikit-learn 1.8.0, imbalanced-learn 0.14.2, xgboost 3.3.0, and scipy 1.17.1, and requirements.txt in the repository pins those versions.

References:

Becker, B. and Kohavi, R. (1996). Adult. UCI Machine Learning Repository. <https://archive.ics.uci.edu/dataset/2/adult>
 Kaggle mirror of the same extract, which is where I pulled the CSV from: <https://www.kaggle.com/datasets/uciml/adult-census-income>
 Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, 2825-2830. <https://scikit-learn.org>
 Lemaitre, G., Nogueira, F., and Aridas, C. (2017). Imbalanced-learn. JMLR 18(17), 1-5. <https://imbalanced-learn.org>
 Chawla, N., Bowyer, K., Hall, L., and Kegelmeyer, W. (2002). SMOTE: Synthetic Minority Over-sampling Technique. JAIR 16, 321-357.
 Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD '16. <https://xgboost.readthedocs.io>
 Breiman, L. (2001). Random Forests. Machine Learning 45, 5-32, and the scikit-learn permutation importance notes at https://scikit-learn.org/stable/modules/permutation_importance.html
 Dietterich, T. (1998). Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms. Neural Computation 10(7), 1895-1923.
 Efron, B. and Tibshirani, R. (1993). An Introduction to the Bootstrap.
 scipy.stats for the t-test, Wilcoxon signed-rank test, and exact binomial test: <https://docs.scipy.org/doc/scipy/reference/stats.html>
 My own earlier milestones in this project, which this report carries forward: the database setup and initial data look, the EDA and feature engineering milestone, and the baseline model milestone, all in <https://github.com/ErikHerb/MSDS640-coursework>

How I used an LLM for this assignment

I used an LLM to help debug my code and troubleshoot errors, including finding a bug where all three of my pipelines were sharing one scaler object, which was quietly changing my XGBoost results. It helped me format the output of my code so the tables printed the way I wanted them to, and it helped me build some of the plots in the notebook. I used it to think through which statistical test fits which comparison, in particular why McNemar's test is the right one for two classifiers on a shared test set and why the Wilcoxon test bottoms out at 0.0625 with five folds. I also used it to check my work against the assignment instructions to make sure I was not missing a required section. I wrote and edited the analysis myself and confirmed every number by running the notebook top to bottom.